Oracle SQL Code Review & Query Logic Validation

Query Review 1

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
ON e.employee_id = d.department_id;
```

Syntax Validation

The SQL syntax is valid.
The query executes successfully and returns rows.

Result Analysis

The query returned employee names with department names. However, the returned data is not reliable because the join condition is incorrect.

Business Logic Issue

The query joins:

e.employee_id = d.department_id

This is not a valid business relationship.

`employee_id` identifies an employee, while `department_id` identifies a department. These two columns should not be matched together. Any matching rows are accidental, not meaningful.

Correct Query

```
SELECT e.first_name,
    e.last_name,
    d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;
```

Reasoning

The correct relationship is between the employee's department ID and the department table's department ID.

employees.department_id = departments.department_id

This uses the foreign key relationship correctly and returns the real department for each employee.

Syntax Validation

The SQL syntax is written correctly, but the query fails during execution.

Error Returned

ORA-01722: invalid number

Result Analysis

The query does not return valid results because Oracle attempts to compare a numeric column with a character column.

Business Logic Issue

The join condition is incorrect:

e.employee_id = j.job_id

`employee_id` is a number, for example:

100, 101, 102

But `job_id` is text, for example:

AD_PRES, IT_PROG, SA_REP

This causes a data type mismatch and also does not represent the correct relationship between employees and jobs.

Correct Query

```
SELECT e.first_name,
    e.last_name,
    j.job_title
FROM employees e
JOIN jobs j
ON e.job_id = j.job_id;
```

Reasoning

The correct business relationship is:

employees.job_id = jobs.job_id

Each employee has a `job_id`, and the `jobs` table stores the details of each job. This query correctly shows each employee with their actual job title.

Query Review 3

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';
```

Syntax Validation

The SQL syntax is valid.
The query executes successfully and returns employees from the Sales department.

Result Analysis

The query returned Sales department employees. The output is correct for showing only Sales employees.

However, the query design is not ideal because it uses a `LEFT JOIN` but then filters the right table in the `WHERE` clause.

Business Logic Issue

This condition:

WHERE d.department_name = 'Sales'

removes all rows where `d.department_name` is `NULL`.

That means the `LEFT JOIN` no longer behaves like a real left join. It behaves like an `INNER JOIN`.

Better Correct Query

```
SELECT e.first_name,
    d.department_name
FROM employees e
JOIN departments d
```

```
ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';
```

Reasoning

If the business requirement is:

Show only employees who work in the Sales department.

Then `INNER JOIN` is better because only matching employees and departments are needed.

The query is clearer, easier to maintain, and avoids misleading future developers.

Query Review 4

```
SELECT e.first_name, m.first_name AS manager_name
FROM employees e
JOIN employees m
ON e.employee_id = m.manager_id;
```

Syntax Validation

The SQL syntax is valid.
The query executes successfully and returns rows.

Result Analysis

The query returned **50 rows**. However, the result is not showing the correct employee-manager relationship.

This result is misleading because the query is matching an employee's ID with another employee's `manager_id`.

Business Logic Issue

The join condition is incorrect:

ON e.employee_id = m.manager_id

This means:

> Show employees who are managers, and show the employees who report to them.

But selected columns make it look like:

employee name | manager name

Actually, in query:

- `e` represents the manager.
- `m` represents the employee working under that manager.

So the aliases are reversed and confusing.

Correct Query

If the requirement is:

> Show each employee with their manager name.

```
SELECT e.first_name AS employee_name,
    m.first_name AS manager_name
FROM employees e
JOIN employees m
ON e.manager_id = m.employee_id;
```

Why This Is Correct

In the HR schema, the employee table has a self-relationship:

employees.manager_id = employees.employee_id

This means:

- `e.manager_id` stores the employee's manager ID.
- `m.employee_id` stores the manager's employee ID.

So the correct relationship is:

ON e.manager_id = m.employee_id

Not:

ON e.employee_id = m.manager_id

Reason

Each employee stores their manager ID in `e.manager_id`.